

2. Skip List – Fundamentals

```
#define MAX_LEVEL 3
```

1. The Ceiling: `#define MAX_LEVEL 3`

This is a rule for the compiler. It says: "Any time you see the word `MAX_LEVEL`, replace it with the number 3."

- *As we start counting at Level 0, `MAX_LEVEL 3` means nodes can exist on Level 0, Level 1, Level 2, or Level 3.*
- *Without a limit, the coin – flip function might flip 100 heads in a row, building a massive 100 – level tower for a single number, which would waste a huge amount of memory.*

```
struct Node
{
    int key;
    // Array of pointers to next nodes, one for each level
    struct Node **forward;
};
```

- *`int key`; This is the actual data we are saving (e. g., the number 42).*
- *`struct Node ** forward`; This is the most complex line in the program.*

In a standard linked list, we just have `struct Node * next` (one pointer to the next node).

But in a Skip List, a node might be on Level 0, Level 1, and Level 2 simultaneously. Therefore, it needs multiple pointers.

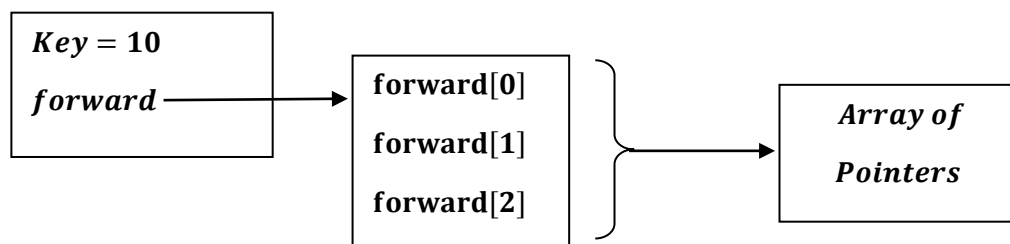
- The double asterisk (**) tells C++ to treat forward as a dynamic array of pointers.

If the keys are 10, 20, 30, 40 then there are exactly :

1 × Node 10
1 × Node 20
1 × Node 30
1 × Node 40

A total of 4 node objects.

Now, say Node 10, it has :



Level 2: [Node 10] -----> [Node 40] -> NULL

Level 1: [Node 10] -----> [Node 30] ----> [Node 40] -> NULL

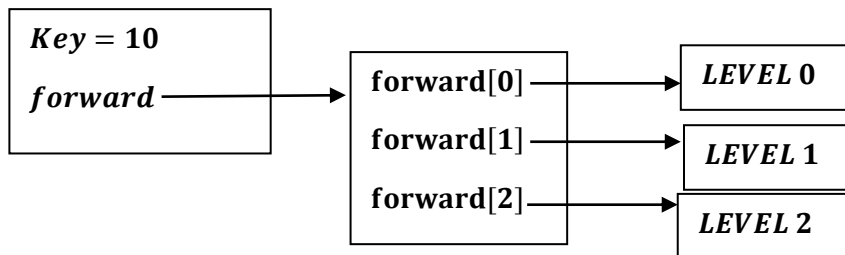
Level 0: [Node 10] ---> [Node 20] -> [Node 30] ---> [Node 40] -> NULL

This does not mean there are three copies of Node 10.

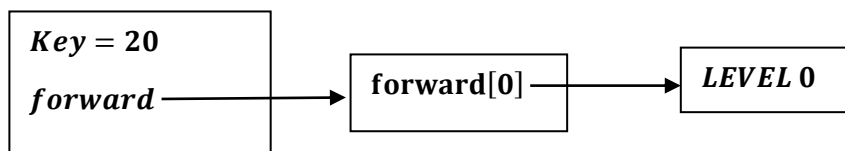
Keys and its corresponding Nodes are arranged in random levels suppose:

<i>Key</i>	<i>Level</i>
10	2
20	0
30	1
40	2

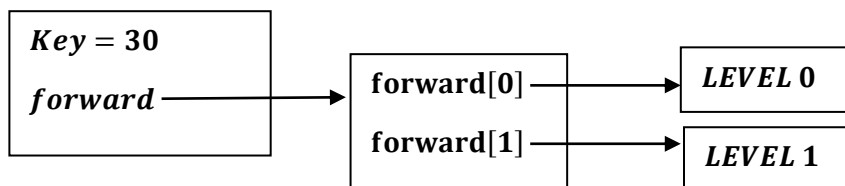
Node 10 , it has :



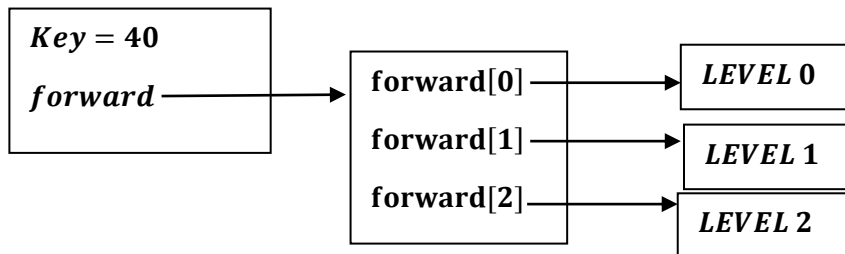
Node 20 , it has :



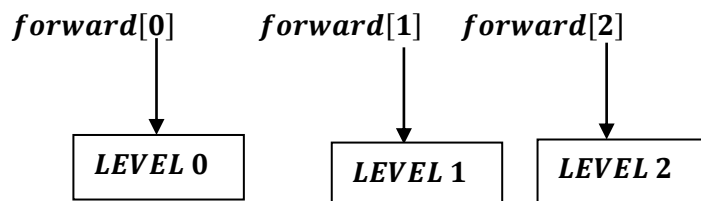
Node 30 , it has :



Node 40 , it has :



Note , Level is by default : MAX_LEVEL = 3. [We will know it during create node module.]



forward[0], here `0` means Level 0 .

forward[1], here `1` means Level 1 , It means Level 0 i.e. forward[0] also present.

forward[2], here `2` means Level 2, It means Level 1 and Level 0 also present, i.e. forward[0], forward[1] also present .

```
// Skip List structure
struct SkipList
{
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};
```

Here `level` is the current maximum level and during creation of list it is assigned to 0 and is not define MAX_LEVEL 3 .

```
lst->level = 0;
```

Now, header pointer:

```
struct Node *header; // Pointer to the header node
```

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level + 1));
```

And

```
lst->header = createNode(-1, MAX_LEVEL);
```

These shows :

header = createNode(-1, MAX_LEVEL = 3)

*forward = (Struct Node **)malloc(sizeof(struct Node *) * (level = 3 + 1));*

or,

*forward = (Struct Node **)malloc(sizeof(struct Node *) * (level = 4));*

∴ At creation of header , we get:

header → *forward*[3] [LEVEL 3] – 8 bytes

4 bytes * 4 = 16 bytes according to 32 bit architecture.

Level 0: [Node 10] ---> [Node 20] -> [Node 30] ---> [Node 40] -> NULL

LEVEL 3 is not reached, therefore header → forward[3] points to NULL.

```
for (int i = 0; i <= level; i++)
{
    n->forward[i] = NULL;
}
```

forward[3] [LEVEL 3] = NULL

During creation of the list.

Header is a pointer variable that points to a special node called the header node (also called a head node or sentinel node) of the skip list.

```
struct SkipList
{
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};
```

- *struct Node *header; means:*
 - *header is a variable.*
 - *Its type is pointer to struct Node.*
 - *It stores the memory address of the header node.*

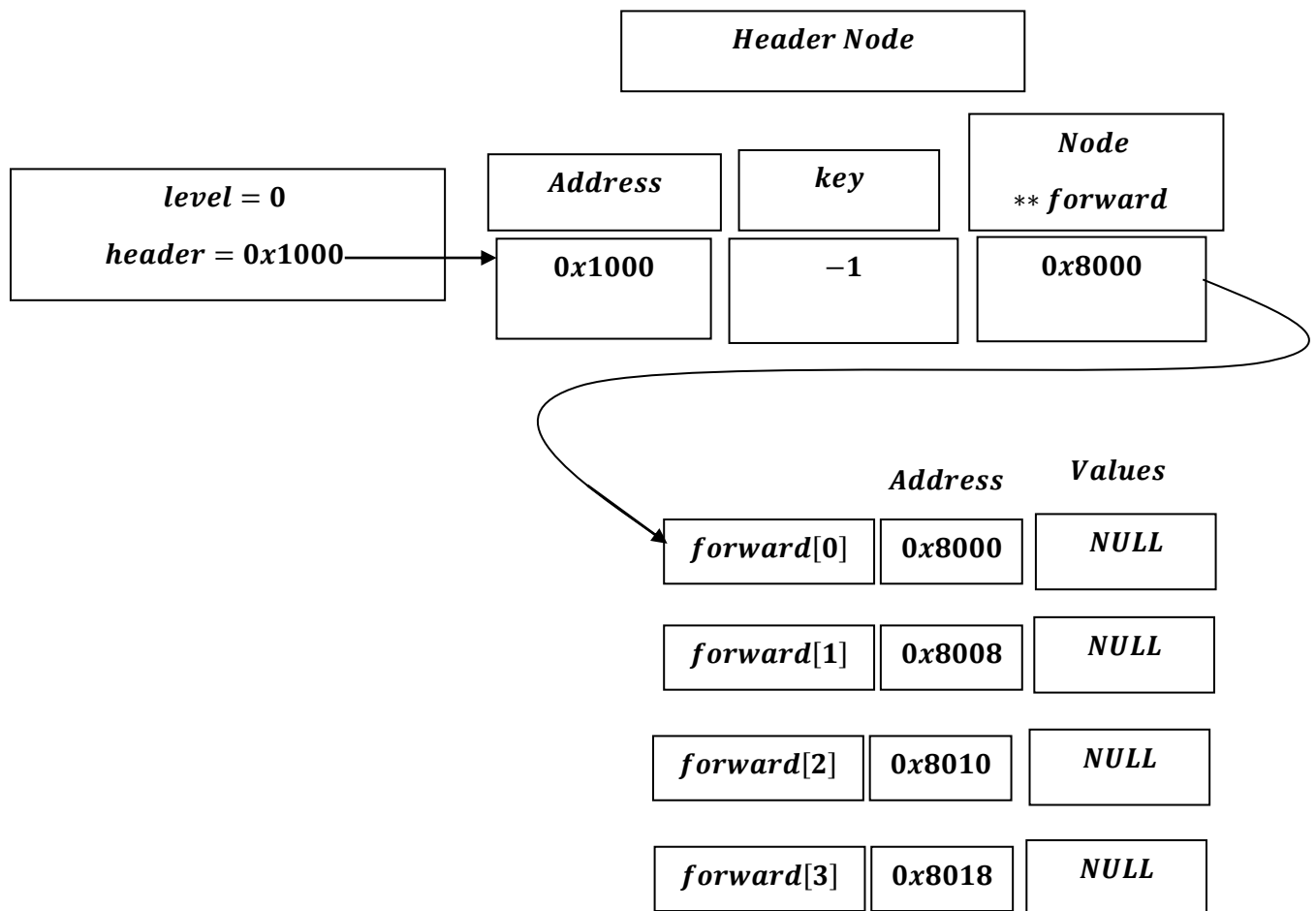
Where is header initialized?

In createSkipList():

```
lst->header = createNode(-1, MAX_LEVEL);
```

Suppose the header node is allocated at address 0x1000.

Then:



So header simply stores the address 0x1000.

key = -1[Dummy value]

This value is just a placeholder. We never search for or display the header's key.

The header exists so that every level has a starting point.

Initially:

Level 3:

Header --forward[3] --> NULL

Level 2:

Header --forward[2] --> NULL

Level 1:

Header --forward[1] --> NULL

Level 0:

Header --forward[0] --> NULL

From here we can get some picture clarity about the program.
